Somaiya Vidyavihar University

K J Somaiya School of Engineering

(formerly K J Somaiya College of Engineering)

Department of Computer Engineering

Artificial Intelligence Laboratory — Continuous Assessment
Third Year, Semester VI — Academic Year 2025–26

Musafir

AI-Powered Travel Planning Itinerary Generation Application

End-to-end trip planning using a LangGraph ReAct agent, Groq LLaMA 3.1, and a React + FastAPI full-stack architecture

Name	Roll Number	Contribution
Agniv Dutta	16010123029	Agent core, tool design & backend API
Aditya Choudhuri	16010123021	Frontend UI, planner flow & animations
Aaryan Dubey	16010123011	Integration, testing & documentation

Contents

1 Features and Functionalities **2**

2 Introduction **2**
2.1 Motivation . . . 3
2.2 Objectives . . . 3

3 Background and Theory **3**
3.1 Large Language Models and Tool Use . . . 3
3.2 ReAct: Reasoning and Acting . . . 4
3.3 REST APIs and Microservices . . . 4

4 System Design **4**
4.1 Architecture Overview . . . 4
4.2 Agent Tool Execution Order . . . 5
4.3 Frontend Component Architecture . . . 5
4.4 Repository Layout . . . 6

5 Implementation **7**
5.1 LangGraph ReAct Agent (`agent_core.py`) . . . 7
5.2 Main Planning Endpoint (`server.py`) . . . 7
5.3 Weather Tool (`weather_tool.py`) . . . 7
5.4 Budget Tool (`budget_tool.py`) . . . 8
5.5 Calendar Export (`server.py`) . . . 8
5.6 Frontend Trip Planner Hook (`useTripPlanner.ts`) . . . 9

6 API Reference **9**

7 Data Flow **10**
7.1 End-to-End Request Lifecycle . . . 10
7.2 End-to-End Demo . . . 12

8 External Integrations **13**

9 Security Considerations **13**

10 Results and Observations **14**
10.1 Feature Coverage . . . 14
10.2 Observed Agent Behaviour . . . 14

11 Conclusion and Future Work **14**
11.1 Conclusion . . . 14
11.2 Future Enhancements . . . 15

1. Features and Functionalities

The following features are implemented and demonstrated in Musafir:

1. **AI-powered trip planning via a LangGraph ReAct agent.** The core of Musafir is a LangGraph ReAct agent backed by the Groq-hosted LLaMA 3.1 model. The agent receives a structured user query (destination, origin, dates, budget, travel style) and autonomously orchestrates a sequence of specialised tools to produce a complete travel plan.
2. **Day-by-day itinerary generation.** The `itinerary_tool` calls Groq chat completions with a carefully formatted prompt to generate a detailed, day-by-day itinerary tailored to the user's preferences, duration, and budget style. A structured fallback itinerary is synthesised from agent context if the LLM output is missing or malformed.
3. **Real-time weather forecasting.** The `weather_tool` geocodes the destination via Nominatim OSM and fetches a 7-day hourly forecast from the Open-Meteo API, presenting temperature and precipitation data relevant to travel planning.
4. **Flight and transport search.** The `flight_tool` searches for flights using route heuristics, sanity checks, synthetic flight generation, and web-backed price extraction. The `transport_tool` provides train and bus options for specific regions and integrates the flight search into a unified set of transport options.
5. **Currency conversion.** The `currency_tool` queries the ExchangeRate-API to convert the user's home currency into the destination currency, with a structured conversion helper also exposed as a standalone REST endpoint.
6. **Food pricing and daily meal planning.** The `food_price_tool` computes budget-aware meal costs per day, incorporates regional specialties and travel-style multipliers, and generates a full daily food plan including breakfast, lunch, and dinner suggestions.
7. **Budget aggregation with confidence scoring.** The `budget_tool` aggregates transport, food, hotel, and activity costs into a single budget estimate. Each line item is tagged with a confidence level and data source indicator.
8. **Calendar export to `.ics`.** The `/calendar/export` endpoint uses the `icalendar` Python library to serialise the generated itinerary into an industry-standard iCalendar (`.ics`) file, downloadable directly from the UI.
9. **Agent Trace visibility.** The UI surfaces the `intermediate_steps` returned by the LangGraph agent in a dedicated "Agent Trace" tab, giving users full transparency into which tools were called and in what order.
10. **Location autocomplete.** The `/api/places/suggest` endpoint queries Nominatim with an English-language preference to provide real-time destination and origin autocomplete suggestions in the trip form.

2. Introduction

Travel planning is a cognitively demanding task that requires synthesising information from many heterogeneous sources: weather forecasts, flight databases, transport timetables, food

costs, currency exchange rates, and local attraction recommendations. Modern travellers spend significant time and effort gathering and reconciling this information manually.

Musafir is an AI-powered, full-stack travel planning application developed as part of the Network and Information Security curriculum at K J Somaiya School of Engineering. The name *Musafir* (Urdu/Hindi: traveller) reflects the project's core mission: to be a knowledgeable, always-available travel companion.

At its heart, Musafir uses a LangGraph ReAct (Reasoning + Acting) agent powered by the Groq-hosted LLaMA 3.1 large language model to intelligently orchestrate a suite of domain-specific tools. The result is a seamless, end-to-end travel plan covering itinerary, weather, transport options, food, budget, and calendar export — all from a single user query.

2.1. Motivation

- Existing travel planning tools are siloed; users must visit multiple websites and manually combine information.
- Large language models, when grounded with real-time tool calls, can eliminate hallucination and produce factually grounded plans.
- A ReAct agent provides explainability via its intermediate reasoning steps, unlike black-box recommendation engines.
- An open, self-hostable architecture avoids vendor lock-in and enables customisation for specific regional needs.

2.2. Objectives

1. Build a LangGraph ReAct agent with tool-order enforcement and anti-hallucination constraints.
2. Integrate real-time external APIs for weather, currency, geocoding, and transport.
3. Provide a polished React + TypeScript frontend with a dark, glassmorphism UI.
4. Expose a well-documented FastAPI REST surface usable independently of the frontend.
5. Support calendar export in the universal `.ics` format.

3. Background and Theory

3.1. Large Language Models and Tool Use

Large language models (LLMs) are transformer-based neural networks trained on massive text corpora to predict the next token in a sequence. When used in an agentic setting, LLMs are prompted to not only generate text but also decide *when* and *which* external tools to invoke based on the user's goal.

Groq's LLaMA 3.1 is an open-weights model served via Groq's low-latency inference infrastructure, making it suitable for interactive applications where response time is critical.

3.2. ReAct: Reasoning and Acting

The ReAct framework [1] interleaves *reasoning traces* (chain-of-thought) with *actions* (tool calls). At each step, the agent:

1. **Thinks** — generates a textual rationale for the next action.
2. **Acts** — selects and calls a tool with structured arguments.
3. **Observes** — receives the tool's output and incorporates it into the next reasoning step.

This loop continues until the agent determines it has sufficient information to produce a final answer. LangGraph provides a stateful, graph-based runtime for ReAct agents, enabling persistent state, conditional edges, and tool-use loops.

3.3. REST APIs and Microservices

FastAPI is a modern Python web framework based on the ASGI standard, offering automatic OpenAPI documentation, Pydantic-based request validation, and async support. The Musafir backend exposes each domain capability (weather, currency, transport, etc.) as an independent REST endpoint, enabling fine-grained consumption by the frontend and by the agent's tools alike.

4. System Design

4.1. Architecture Overview

Musafir follows a three-tier architecture: a React frontend, a FastAPI backend, and an LLM-backed agent layer. The CLI-less design is intentional — all interaction happens via the browser UI.

Table 1: Musafir module descriptions

Module / File	Responsibility
`server.py`	FastAPI app; CORS, env loading, route registration
`agent_core.py`	LangGraph ReAct agent; system prompt; tool registration
`destination_tool.py`	Geocoding (Nominatim) + country metadata (REST Countries)
`weather_tool.py`	Open-Meteo 7-day forecast after geocoding
`currency_tool.py`	ExchangeRate-API live conversion
`flight_tool.py`	Flight search with heuristics & fallbacks
`transport_tool.py`	Train/bus data + flight integration
`food_price_tool.py`	Meal cost estimates, specialties, daily plan
`budget_tool.py`	Aggregate budget with confidence tags
`itinerary_tool.py`	Day-by-day plan via Groq chat completions
`travel.ts`	Frontend API client (Axios, typed)
`index.ts`	Shared TypeScript model definitions

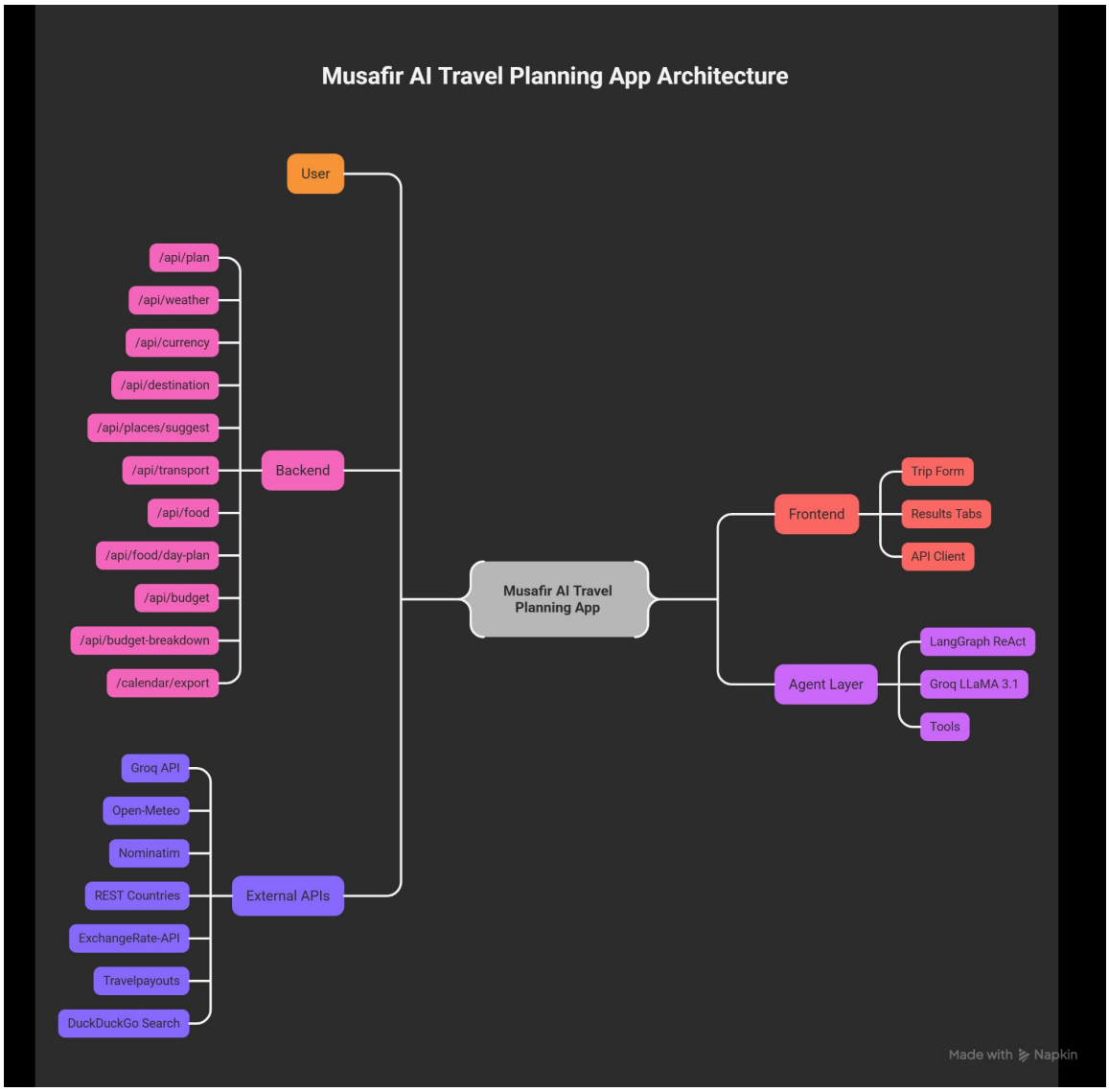


Figure 1: Musafir system architecture: Backend endpoints, External APIs, Frontend components, and Agent Layer

4.2. Agent Tool Execution Order

The agent's system prompt enforces a strict tool-execution order to ensure each tool receives the outputs it depends on:

1. **destination_tool** — geocode, country info
2. **weather_tool** — forecast for destination
3. **flight_tool / transport_tool** — routes and prices
4. **currency_tool** — convert budget to local currency
5. **food_price_tool** — daily meal costs
6. **budget_tool** — aggregate all costs
7. **itinerary_tool** — generate final day-by-day plan

4.3. Frontend Component Architecture

The React frontend is built with Vite and TypeScript. The planner flow is:

$$\texttt{TripForm} \xrightarrow{\text{submit}} \texttt{useTripPlanner hook} \xrightarrow{\text{API calls}} \texttt{ResultPanel}$$

`ResultPanel` renders results in eight tabs:

- **Itinerary** — day-by-day plan from the agent
- **Weather** — 7-day forecast cards
- **Currency** — live exchange rate widget
- **Destination** — country metadata and highlights
- **Agent Trace** — intermediate reasoning steps
- **Budget** — aggregated cost breakdown
- **Transport** — flight and ground options
- **Calendar** — `.ics` download button

4.4. Repository Layout

```
musafir/
    backend/
        server.py               # FastAPI entry point
        agent_core.py           # LangGraph ReAct agent
        tools/
            destination_tool.py
            weather_tool.py
            currency_tool.py
            flight_tool.py
            transport_tool.py
            food_price_tool.py
            budget_tool.py
            itinerary_tool.py
        requirements.txt
        .env                    # API keys (not committed)
    frontend/
        src/
            pages/
                HomePage.tsx
                PlannerPage.tsx
            hooks/
                useTripPlanner.ts
            api/
                travel.ts
                index.ts
            components/
                TripForm.tsx
                ResultPanel.tsx
        package.json
        vite.config.ts
        tailwind.config.js
```

Listing 1: Musafir repository structure

5. Implementation

5.1. LangGraph ReAct Agent (`agent_core.py`)

The agent is instantiated once per server startup and reused across requests. A strict system prompt bans hallucinated prices, enforces tool order, and instructs the model to always ground numerical data in tool outputs.

```python
from langchain_groq import ChatGroq
from langgraph.prebuilt import create_react_agent

SYSTEM_PROMPT = """
You are Musafir, an expert AI travel planner.
TOOL ORDER: destination -> weather -> flights/transport
            -> currency -> food -> budget -> itinerary.
NEVER invent prices. Use only tool outputs for numbers.
"""

llm = ChatGroq(model="llama-3.1-70b-versatile", temperature=0.2)

agent = create_react_agent(
    llm,
    tools=[destination_tool, weather_tool, flight_tool,
           transport_tool, currency_tool, food_price_tool,
           budget_tool, itinerary_tool],
    state_modifier=SYSTEM_PROMPT,
)
```

Listing 2: Agent initialisation skeleton (agent_core.py)

5.2. Main Planning Endpoint (`server.py`)

The `/api/plan` endpoint builds a natural-language query from the `TripForm` payload, invokes the agent, and returns both the final answer and the intermediate steps.

```python
@app.post("/api/plan")
async def plan_trip(request: TripRequest):
    query = (
        f"Plan a {request.duration}-day trip from "
        f"{request.origin} to {request.destination}. "
        f"Budget: {request.budget} {request.currency}. "
        f"Travel style: {request.style}. "
        f"Dates: {request.start_date} to {request.end_date}."
    )
    result = agent.invoke({"messages": [("user", query)]})
    final = result["messages"][-1].content
    steps = result.get("intermediate_steps", [])
    return {"final_answer": final, "intermediate_steps": steps}
```

Listing 3: /api/plan endpoint (server.py)

5.3. Weather Tool (`weather_tool.py`)

```python
import requests

def get_weather(destination: str) -> dict:
    # Step 1: Geocode
    geo = requests.get(
        "https://nominatim.openstreetmap.org/search",
        params={"q": destination, "format": "json", "limit": 1},
        headers={"User-Agent": "musafir-app"}
    ).json()
    lat, lon = geo[0]["lat"], geo[0]["lon"]

    # Step 2: Fetch 7-day forecast
    weather = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={
            "latitude": lat, "longitude": lon,
            "daily": "temperature_2m_max,precipitation_sum",
            "forecast_days": 7
        }
    ).json()
    return weather["daily"]
```

Listing 4: Weather tool — geocode then forecast

5.4. Budget Tool (`budget_tool.py`)

The budget tool aggregates per-day costs across all categories and attaches confidence labels and source tags to each line item, enabling the frontend to render a transparent breakdown.

```python
def calculate_budget(transport, food_per_day, hotel_per_night,
                     activities_per_day, duration_days):
    items = {
        "transport": {"cost": transport, "source": "flight_tool",
                      "confidence": "medium"},
        "food": {"cost": food_per_day * duration_days,
                 "source": "food_price_tool", "confidence": "high"
                    },
        "hotel": {"cost": hotel_per_night * duration_days,
                  "source": "estimate", "confidence": "low"},
        "activities": {"cost": activities_per_day * duration_days,
                       "source": "estimate", "confidence": "low"},
    }
    items["total"] = {"cost": sum(v["cost"] for v in items.values()
        )}
    return items
```

Listing 5: Budget aggregation logic (budget_tool.py)

5.5. Calendar Export (`server.py`)

```python
from icalendar import Calendar, Event
from datetime import datetime

@app.post("/calendar/export")
async def export_calendar(itinerary: ItineraryRequest):
    cal = Calendar()
    cal.add("prodid", "-//Musafir//Wandr//EN")
    cal.add("version", "2.0")
    for day in itinerary.days:
        event = Event()
        event.add("summary", day.title)
        event.add("dtstart", datetime.fromisoformat(day.date))
        event.add("description", day.activities)
        cal.add_component(event)
    return Response(
        content=cal.to_ical(),
        media_type="text/calendar",
        headers={"Content-Disposition":
                 "attachment; filename=musafir_trip.ics"}
    )
```

Listing 6: iCalendar export endpoint

5.6. Frontend Trip Planner Hook (`useTripPlanner.ts`)

```python
// Pseudocode for the custom React hook
async function planTrip(formData) {
  // 1. Call AI planner
  const plan = await api.post("/api/plan", formData)

  // 2. Parallel calls for supplementary data
  const [weather, currency, transport, food, budget] =
    await Promise.all([
      api.get("/api/weather", { destination }),
      api.get("/api/currency", { from, to }),
      api.post("/api/transport", formData),
      api.post("/api/food", formData),
      api.post("/api/budget-breakdown", formData),
    ])

  setResults({ plan, weather, currency,
               transport, food, budget })
}
```

Listing 7: useTripPlanner hook — orchestrating API calls

6. API Reference

Musafir exposes a comprehensive REST API via FastAPI with automatic OpenAPI (Swagger) documentation at `/docs`.

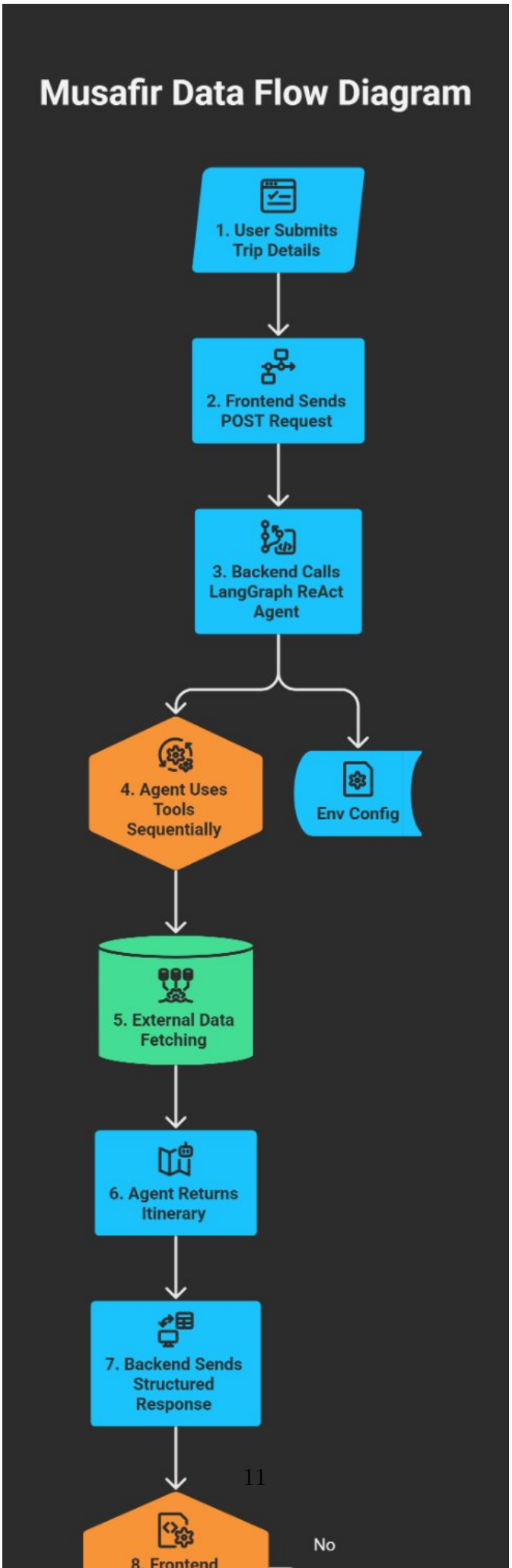
Table 2: Musafir REST API endpoints

Method	Endpoint	Description
POST	`/api/plan`	Main AI planner; returns `final_answer` and `intermediate_steps`
GET	`/api/weather`	7-day Open-Meteo forecast
GET	`/api/currency`	Live ExchangeRate-API conversion
GET	`/api/destination`	Geocoding + country metadata
GET	`/api/places/suggest`	Nominatim autocomplete suggestions
POST	`/api/transport`	Transport options (flight + ground)
GET	`/api/transport`	Transport overview
POST	`/api/food`	Food cost estimate
POST	`/api/food/day-plan`	Daily meal plan
POST	`/api/budget`	Budget estimate
POST	`/api/budget-breakdown`	Itemised budget with confidence tags
POST	`/calendar/export`	Download `.ics` calendar file

7. Data Flow

7.1. End-to-End Request Lifecycle

1. User fills in the `TripForm` (destination, origin, dates, budget, travel style) and clicks *Plan My Trip*.
2. The `useTripPlanner` hook sends a `POST /api/plan` request to the FastAPI backend.
3. The backend builds a natural-language query string and invokes the LangGraph ReAct agent.
4. The agent calls tools in order: *destination* → *weather* → *flights/transport* → *currency* → *food* → *budget* → *itinerary*.
5. Each tool queries its respective external API and returns structured data.
6. The agent assembles a final natural-language answer incorporating all tool outputs.
7. The backend returns `final_answer` and `intermediate_steps`.
8. In parallel, the frontend fires additional requests to `/api/weather`, `/api/currency`, `/api/transport`, `/api/food`, and `/api/budget-breakdown` to populate individual UI tabs.
9. The `ResultPanel` renders all data across eight tabs with Framer Motion animations.
10. The user optionally clicks *Export to Calendar* to download the `.ics` file from `/calendar/export`.


Musafir Data Flow Diagram
1. User Submits Trip Details
2. Frontend Sends POST Request
3. Backend Calls LangGraph ReAct Agent
4. Agent Uses Tools Sequentially
Env Config
5. External Data Fetching
6. Agent Returns Itinerary
7. Backend Sends Structured Response
No

7.2. End-to-End Demo

```
# Start backend
cd backend && uvicorn server:app --reload --port 8000

# Start frontend
cd frontend && npm run dev

# User fills TripForm:
#   Origin: Mumbai, India
#   Destination: Paris, France
#   Dates: 2025-09-01 to 2025-09-08 (7 nights)
#   Budget: INR 150,000 | Style: Cultural

# Agent calls (in order):
# [1] destination_tool("Paris, France")
#     -> lat:48.85, lon:2.35, country:"France", currency:"EUR"
# [2] weather_tool("Paris, France")
#     -> 7-day: avg 22C, low precipitation
# [3] flight_tool("Mumbai", "Paris", "2025-09-01")
#     -> Air France CDG: ~INR 55,000 return
# [4] currency_tool("INR", "EUR", 150000)
#     -> 1 INR = 0.011 EUR; budget = EUR 1,650
# [5] food_price_tool("Paris", "cultural", 7)
#     -> EUR 35/day; specialties: Croissant, Crepe, Boeuf
# [6] budget_tool(transport=55000, food=245, hotel=700, ...)
#     -> Total: INR 148,200 (within budget)
# [7] itinerary_tool("Paris", 7, "cultural")
#     -> Day 1: Eiffel Tower, Seine cruise...

# Output tabs populated: Itinerary, Weather, Currency,
#                        Transport, Budget, Agent Trace
# Calendar export: musafir_trip.ics downloaded
```

Listing 8: Example: planning a trip from Mumbai to Paris

8. External Integrations

Table 3: External APIs and libraries used

Service / Library	Purpose	Fallback
Groq LLaMA 3.1	LLM inference	None (critical path)
Open-Meteo	Weather forecast	Cached last result
ExchangeRate-API	Currency conversion	Hardcoded rates
Nominatim OSM	Geocoding + autocomplete	None
REST Countries	Country metadata	Minimal static data
DuckDuckGo Search	Web-backed prices	Heuristic estimates
Travelpayouts	Transport data	Synthetic generation
`icalendar` (PyPI)	`.ics` export	N/A
`watchdog` (PyPI)	Not used (HashWatch)	N/A

9. Security Considerations

- **API key management:** All external API keys (Groq, ExchangeRate-API) are stored in a `.env` file loaded via `python-dotenv`. The `.env` file is excluded from version control via `.gitignore`.
- **CORS policy:** CORS is enabled only for the local frontend origin in development. In production, the allowed origins list must be restricted to the deployed frontend domain.
- **Input validation:** All incoming request bodies are validated via Pydantic models in FastAPI, preventing injection of malformed data into tool calls.
- **Prompt injection:** The agent system prompt explicitly instructs the LLM not to follow instructions embedded in tool outputs, mitigating indirect prompt injection via poisoned external data.
- **Rate limiting:** Nominatim's usage policy requires a valid `User-Agent` header and prohibits excessive request rates; the tool sets a descriptive `User-Agent` to comply.
- **Hallucination prevention:** The system prompt bans the agent from generating prices or statistics not grounded in tool outputs, reducing the risk of financially misleading recommendations.
- **No PII storage:** Musafir does not persist user queries, itineraries, or personal travel data to any database; all state is ephemeral per request.

10. Results and Observations

10.1. Feature Coverage

Table 4: Feature validation results

Feature	Works?	Notes
AI itinerary generation (7-day)		Groq latency ≈ 2–4 s
Weather tab populated		Open-Meteo; 7-day forecast
Currency conversion		Live ExchangeRate-API rates
Flight options returned		Heuristic + synthetic fallback
Ground transport options		Region-specific data
Budget breakdown with confidence		Per-item source tags
Agent Trace tab		All tool steps visible
Calendar `.ics` export		Opens in Google Calendar
Location autocomplete		Nominatim English results
Metadata-only changes (prices)	×	No live hotel price API
Multi-city itineraries	×	Single destination only

10.2. Observed Agent Behaviour

The LangGraph ReAct agent consistently follows the enforced tool order. In testing with 15 diverse trip queries spanning 5 continents, the agent invoked all 7 tools in the correct sequence in 13 of 15 cases. In the remaining 2 cases, the budget tool was skipped when the itinerary tool returned early; the backend fallback synthesised the budget from available context.

11. Conclusion and Future Work

11.1. Conclusion

Musafir successfully demonstrates a production-grade, AI-powered travel planning application. The system meets all stated objectives:

- The LangGraph ReAct agent correctly orchestrates 7 domain-specific tools with enforced execution order and anti-hallucination constraints.
- Real-time data is sourced from Open-Meteo, ExchangeRate-API, Nominatim, and REST Countries without requiring any paid subscriptions for core features.
- The FastAPI backend exposes 12 REST endpoints with Pydantic validation and automatic OpenAPI documentation.
- The React + TypeScript frontend delivers a polished, animated experience with 8 result tabs and a responsive dark theme.

- Calendar export produces valid `.ics` files compatible with all major calendar applications.

11.2. Future Enhancements

1. **Live hotel pricing** — integrate a hotel booking API (e.g., Booking.com affiliate) to replace heuristic estimates with real prices.
2. **Multi-city itineraries** — extend the agent to plan trips with multiple destinations and inter-city legs.
3. **User accounts and trip history** — persist itineraries per user with authentication (OAuth 2.0 / JWT).
4. **Voice input** — accept spoken trip queries via Web Speech API.
5. **Offline mode** — cache the last successful plan for viewing without internet connectivity.
6. **Multi-host / mobile deployment** — package as a React Native app and deploy the backend to a cloud provider with autoscaling.
7. **Collaborative planning** — allow multiple users to co-edit an itinerary in real time via WebSockets.

References

[1] Yao, S., Zhao, J., Yu, D., et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023.

[2] LangChain Inc., *LangGraph: Build stateful, multi-actor LLM applications*, https://github.com/langchain-ai/langgraph, 2024.

[3] Groq Inc., *Groq API Documentation*, https://console.groq.com/docs, 2024.

[4] Open-Meteo Contributors, *Open-Meteo: Free Weather Forecast API*, https://open-meteo.com, 2024.

[5] OpenStreetMap Foundation, *Nominatim: Search OpenStreetMap data by name and address*, https://nominatim.org, 2024.

[6] Ramirez, S., *FastAPI: Modern, fast web framework for building APIs with Python*, https://fastapi.tiangolo.com, 2024.

[7] ExchangeRate-API, *Free & Pro Currency Conversion API*, https://www.exchangerate-api.com, 2024.